

Name: Atharva Humane
PRN: 24070521240

Experiment 1

Title: Demonstrate Inline, Internal, and External JavaScript; Use Console Methods; Create a Webpage Showing User Information and a Welcome Message

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Experiment 1 - JavaScript Demo</title>
  <link rel="stylesheet" href="style.css">

  <!-- Internal JavaScript -->
  <script>
    console.log("Internal JS: Page is loading...");

    function displayWelcome() {
      document.getElementById("welcome").innerText = "Welcome to the User Portal!";
      console.info("Internal JS: Welcome message set on page load.");
    }
  </script>
</head>

<body onload="displayWelcome()">

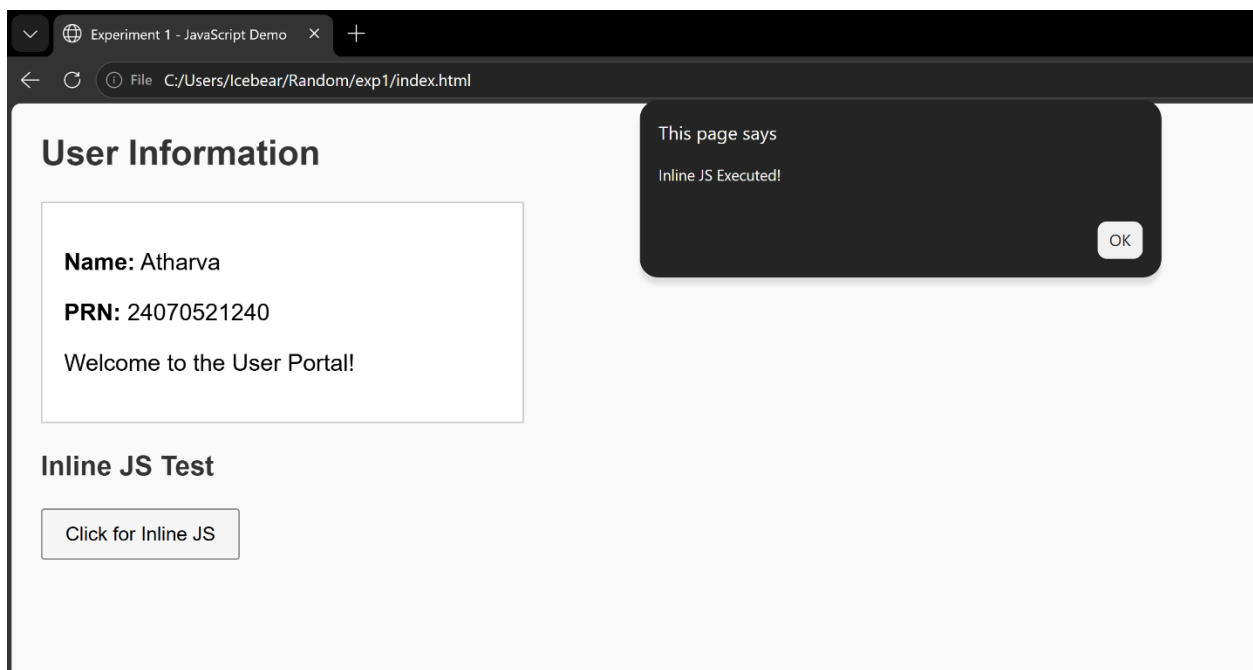
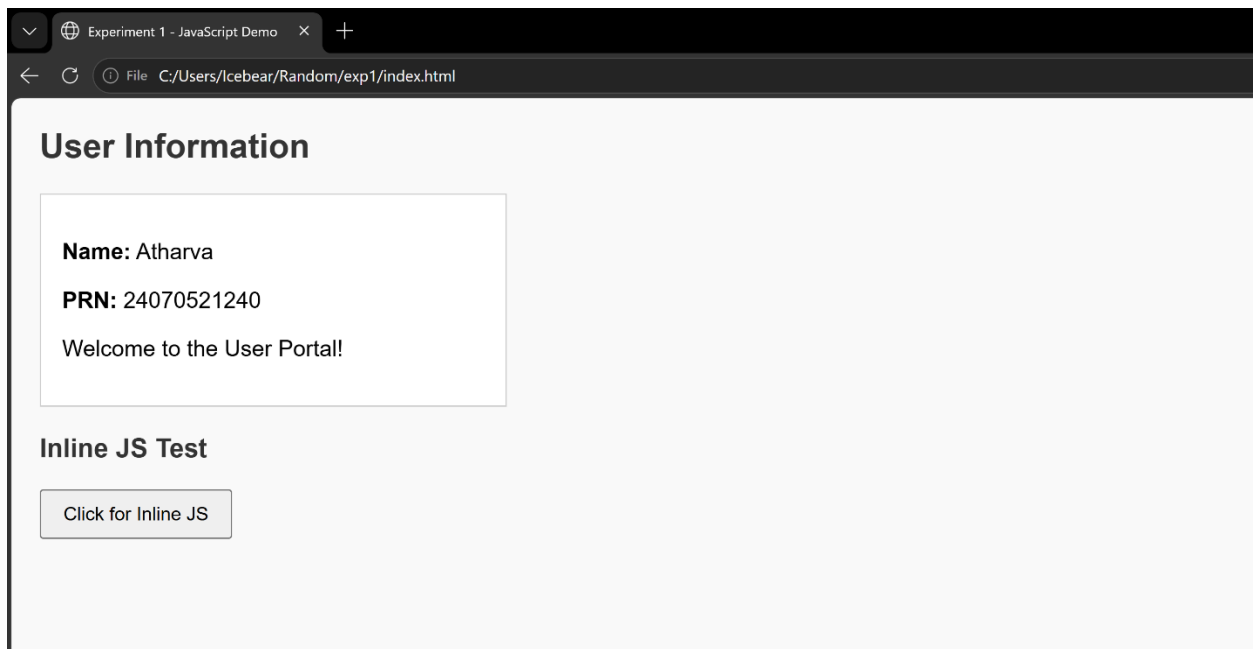
  <h2>User Information</h2>

  <div class="info-box">
    <p><b>Name:</b> Atharva</p>
    <p><b>PRN:</b> 24070521240</p>
    <p id="welcome"></p>
  </div>

  <h3>Inline JS Test</h3>
  <!-- Inline JavaScript -->
  <button onclick="console.warn('Inline JS: Button clicked'); alert('Inline JS Executed!');">
    Click for Inline JS
  </button>

  <!-- External JavaScript -->
  <script src="script.js"></script>
</body>
</html>
```

Output



Result / Conclusion

The experiment was performed successfully. Inline, internal, and external JavaScript were implemented in an HTML webpage. Different console methods were used to display messages for debugging purposes, and the webpage successfully displayed user information along with a personalized welcome message. This experiment demonstrated the different ways of integrating JavaScript into web pages and highlighted the use of console methods during development.

Case Study 1

Title: Create a webpage for the college department and link the Student Information page with it, and write a JavaScript program to display “Welcome to SIT Nagpur” in an alert box.

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Computer Science Department - SIT Nagpur</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f4f4f4;
    }

    h1,
    h2 {
      color: #1a365d;
    }

    .container {
      background-color: #ffffff;
      padding: 20px;
      border-radius: 5px;
      max-width: 700px;
    }

    a {
      display: inline-block;
      margin-top: 15px;
      color: #0066cc;
      text-decoration: none;
      font-weight: bold;
    }

    a:hover {
      text-decoration: underline;
    }
  </style>
  <script>
    alert("Welcome to SIT Nagpur");
  </script>
</head>

<body>

  <div class="container">
    <h1>Department of Computer Science & Engineering</h1>
    <p>PRN: 24070521240</p>

    <h2>About Department</h2>
    <p>The department offers quality education in computer science, focusing on modern computing technologies, software development, and artificial intelligence.</p>

    <h2>Courses Offered</h2>
    <ul>
      <li>B.Tech in Computer Science & Engineering</li>
    </ul>
  </div>
</body>
</html>
```

```

        <h2>Quick Links</h2>
        <p><a href="student.html">Go to Student Information Page &rarr;</a></p>
    </div>

</body>

</html>

<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Student Information - SIT Nagpur</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
            background-color: #f4f4f4;
        }

        h1 {
            color: #1a365d;
        }

        .container {
            background-color: #ffffff;
            padding: 20px;
            border-radius: 5px;
            max-width: 700px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            margin-top: 15px;
        }

        table,
        th,
        td {
            border: 1px solid #ccc;
        }
        th,
        td {
            padding: 10px;
            text-align: left;
        }
        th {
            background-color: #e2e8f0;
        }

        a {
            display: inline-block;
            margin-top: 15px;
            color: #0066cc;
            text-decoration: none;
            font-weight: bold;
        }

        a:hover {
            text-decoration: underline;
        }
    </style>
</head>

<body>
    <div class="container">
        <h1>Student Information</h1>
        <p>Details of registered department students:</p>
        <table>
            <thead>

```

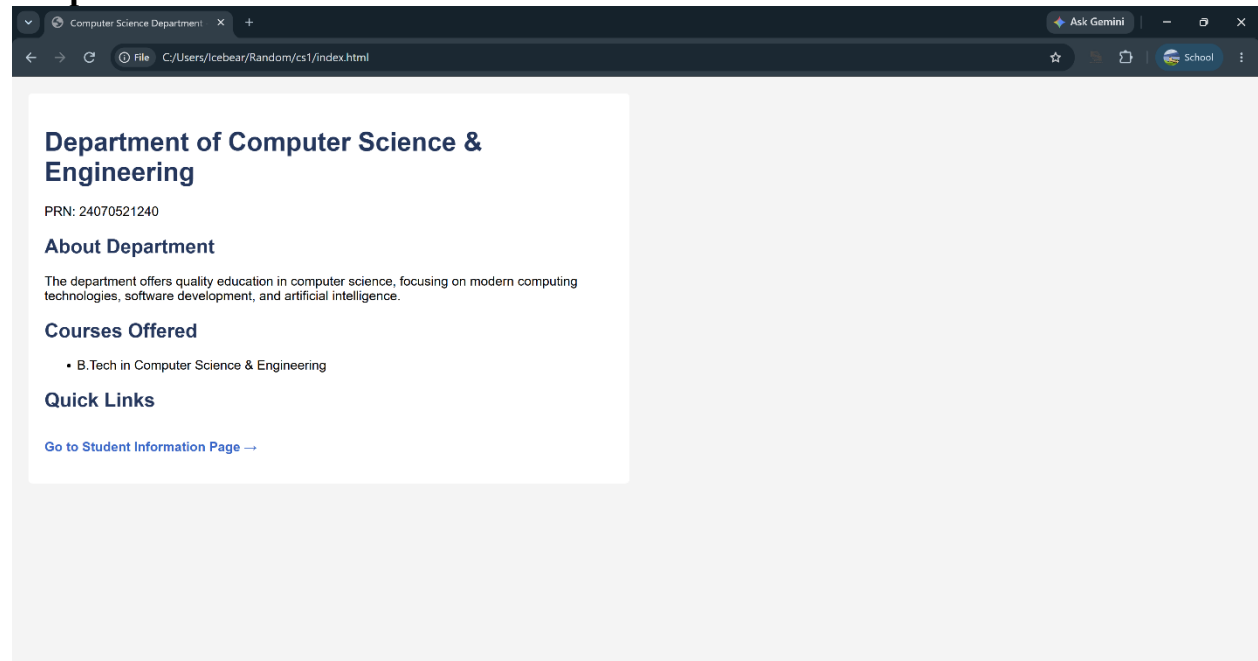
```

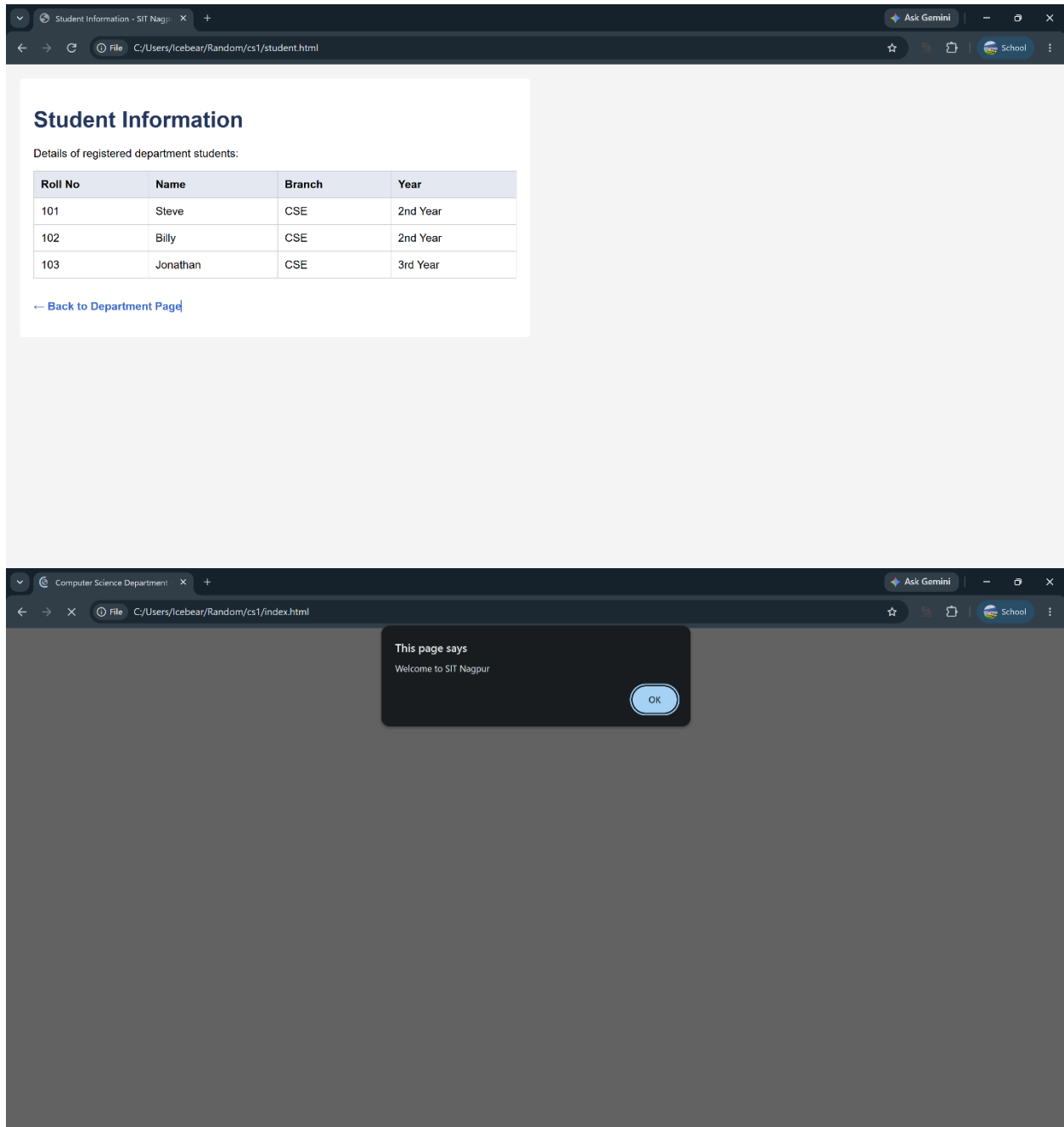
        <tr>
            <th>Roll No</th>
            <th>Name</th>
            <th>Branch</th>
            <th>Year</th>
        </tr>
    </thead>
    <tbody>
        <tr>
            <td>101</td>
            <td>Steve</td>
            <td>CSE</td>
            <td>2nd Year</td>
        </tr>
        <tr>
            <td>102</td>
            <td>Billy</td>
            <td>CSE</td>
            <td>2nd Year</td>
        </tr>
        <tr>
            <td>103</td>
            <td>Jonathan</td>
            <td>CSE</td>
            <td>3rd Year</td>
        </tr>
    </tbody>
</table>

<p><a href="index.html">&larr; Back to Department Page</a></p>
</div>
</body>
</html>

```

Output





Result / Conclusion

The college department webpage was successfully created using HTML. A link to the **Student Information** page was added and tested successfully. JavaScript was also implemented to display “**Welcome to SIT Nagpur**” in an alert box when the webpage is opened.

Experiment 2

Title: Use var, let, const, and Demonstrate Basic Features like Template Literals and Destructuring; Build a Billing Calculator with User Input

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Billing Calculator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 30px auto;
      max-width: 500px;
      background-color: #f4f6f8;
      color: #333;
    }

    .calculator {
      background: #ffffff;
      padding: 24px;
      border: 1px solid #dcdcdc;
      border-radius: 8px;
      box-shadow: 0 2px 5px rgba(0, 0, 0, 0.05);
    }

    h2,
    h3 {
      margin-top: 0;
      color: #222;
    }

    .input-group {
      margin-bottom: 12px;
    }

    label {
      display: block;
      margin-bottom: 4px;
      font-weight: bold;
      font-size: 14px;
    }

    input {
      width: 100%;
      padding: 8px;
      box-sizing: border-box;
      border: 1px solid #ccc;
      border-radius: 4px;
    }

    .btn-add {
```



```

        width: 100%;
        padding: 9px;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-size: 15px;
        margin-top: 6px;
    }

    .btn-add:hover {
        background-color: #0056b3;
    }

    .btn-calc {
        width: 100%;
        padding: 10px;
        background-color: #28a745;
        color: white;
        border: none;
        border-radius: 4px;
        cursor: pointer;
        font-size: 16px;
        margin-top: 15px;
    }

    .btn-calc:hover {
        background-color: #218838;
    }

    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 15px;
    }

    th,
    td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
        font-size: 14px;
    }

    th {
        background-color: #f1f1f1;
    }

    .btn-remove {
        background-color: #dc3545;
        color: white;
        border: none;
        padding: 4px 8px;
        border-radius: 3px;
        cursor: pointer;
    }

    .btn-remove:hover {
        background-color: #c82333;
    }

    .bill-result {
        margin-top: 20px;
        padding: 15px;
        background: #e9ecef;
        border-radius: 6px;
        border-left: 4px solid #28a745;
    }
</style>
</head>

```

```

<body>

  <div class="calculator">
    <h2>Smart Billing Calculator - 24070521240</h2>

    <!-- Input Form -->
    <div class="input-group">
      <label for="itemName">Item Name:</label>
      <input type="text" id="itemName" placeholder="e.g. Notebook">
    </div>

    <div class="input-group">
      <label for="price">Price ($):</label>
      <input type="number" id="price" placeholder="e.g. 15.50" step="0.01">
    </div>

    <div class="input-group">
      <label for="quantity">Quantity:</label>
      <input type="number" id="quantity" placeholder="e.g. 2" min="1">
    </div>

    <button class="btn-add" onclick="addItem()">+ Add Item</button>

    <!-- Added Items Table -->
    <h3>Item List</h3>
    <table>
      <thead>
        <tr>
          <th>Item</th>
          <th>Price</th>
          <th>Qty</th>
          <th>Total</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody id="itemsTableBody">
        <tr>
          <td colspan="5" style="text-align: center; color: #777;">No items added yet.</td>
        </tr>
      </tbody>
    </table>

    <button class="btn-calc" onclick="calculateBill()">Calculate Total Bill</button>

    <!-- Final Bill Receipt -->
    <div id="result" class="bill-result" style="display: none;"></div>
  </div>

  <script>
    // 1. Demonstration of var (Global Store Variables)
    var storeName = "Campus Stationery & Supplies";
    var itemList = [];

    // Add item function
    function addItem() {
      const nameInput = document.getElementById('itemName').value;
      const priceInput = parseFloat(document.getElementById('price').value);
      const qtyInput = parseInt(document.getElementById('quantity').value);

      if (!nameInput || isNaN(priceInput) || isNaN(qtyInput) || qtyInput <= 0) {
        alert("Please enter valid item name, price, and quantity!");
        return;
      }

      // Create item object
      const itemObj = {
        name: nameInput,
        price: priceInput,
        qty: qtyInput
      }
    }
  </script>

```

```

    };

    // Push to global array
    itemList.push(itemObj);

    // Clear inputs
    document.getElementById('itemName').value = '';
    document.getElementById('price').value = '';
    document.getElementById('quantity').value = '';

    // Update UI table
    renderTable();
}

// Render table rows
function renderTable() {
    const tableBody = document.getElementById('itemsTableBody');

    if (itemList.length === 0) {
        tableBody.innerHTML = `<tr><td colspan="5" style="text-align: center; color:
#777;">No items added yet.</td></tr>`;
        return;
    }

    // 2. Demonstration of let (Loop accumulator)
    let tableHTML = "";

    for (let i = 0; i < itemList.length; i++) {
        // 3. Demonstration of Object Destructuring
        const { name, price, qty } = itemList[i];

        let itemTotal = price * qty;

        // 4. Demonstration of Template Literals
        tableHTML += `
            <tr>
                <td>${name}</td>
                <td>${price.toFixed(2)}</td>
                <td>${qty}</td>
                <td>${itemTotal.toFixed(2)}</td>
                <td><button class="btn-remove" onclick="removeItem(${i})">X</button></td>
            </tr>
        `;
    }

    tableBody.innerHTML = tableHTML;
}

// Remove single item
function removeItem(index) {
    itemList.splice(index, 1);
    renderTable();
}

// Calculate final total bill
function calculateBill() {
    if (itemList.length === 0) {
        alert("Add at least one item before calculating!");
        return;
    }

    // 5. Demonstration of const
    const TAX_RATE = 0.10; // 10% Tax
    const DISCOUNT_RATE = 0.05; // 5% Discount

    // 6. Demonstration of Array Destructuring
    const rates = [TAX_RATE * 100, DISCOUNT_RATE * 100];
    const [taxPercent, discountPercent] = rates;

    let subtotal = 0;

```

```

// Calculate subtotal using destructuring inside loop
for (let item of itemsList) {
  const { price, qty } = item;
  subtotal += price * qty;
}

let taxAmount = subtotal * TAX_RATE;
let discountAmount = subtotal * DISCOUNT_RATE;
let grandTotal = subtotal + taxAmount - discountAmount;

// Template literal for Receipt HTML
const billHTML = `
  <h3>${storeName}</h3>
  <p><b>Total Items:</b> ${itemsList.length}</p>
  <hr>
  <p><b>Subtotal:</b> ${subtotal.toFixed(2)}</p>
  <p><b>Tax (${taxPercent}%):</b> +${taxAmount.toFixed(2)}</p>
  <p><b>Discount (${discountPercent}%):</b> -${discountAmount.toFixed(2)}</p>
  <hr>
  <h4><b>Grand Total:</b> ${grandTotal.toFixed(2)}</h4>
`;

const resultDiv = document.getElementById('result');
resultDiv.innerHTML = billHTML;
resultDiv.style.display = 'block';

// Console log using template literal
console.log(`[Bill Generated] ${itemsList.length} items. Grand Total:
${grandTotal.toFixed(2)}`);
}
</script>
</body>
</html>

```

Output

Smart Billing Calculator - 24070521240

Item Name:
e.g. Notebook

Price (\$):
e.g. 15.50

Quantity:
e.g. 2

+ Add Item

Item List

Item	Price	Qty	Total	Action
Notebook	\$10.00	3	\$30.00	X
pen	\$1.00	2	\$2.00	X

Calculate Total Bill

Campus Stationery & Supplies

Total Items: 2

Subtotal: \$32.00

Tax (10%): +\$3.20

Discount (5%): -\$1.60

Grand Total: \$33.60

Result / Conclusion

The experiment was performed successfully. The variables were declared using var, let, and const, and modern JavaScript features such as template literals and destructuring were implemented. A billing calculator was developed to accept user input, calculate the total bill including GST, and display the final amount. This experiment demonstrated the practical use of JavaScript variable declarations and ES6 features in creating interactive web applications.

Case Study 2

Title: Develop the webpage for an online shopping website using HTML,CSS and HTML5 tags And calculate the bill.

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Shopping - Bill Calculator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      margin: 20px;
    }

    .container {
      width: 500px;
      background: #ffffff;
      padding: 20px;
      margin: 0 auto;
      border: 1px solid #ccc;
      box-shadow: 2px 2px 8px rgba(0, 0, 0, 0.1);
    }

    h2,
    h3 {
      text-align: center;
      color: #333;
      margin-top: 5px;
    }

    fieldset {
      margin-bottom: 15px;
      padding: 10px 15px;
      border: 1px solid #ccc;
    }

    legend {
      font-weight: bold;
      font-size: 14px;
    }

    .form-group {
      margin-bottom: 10px;
    }

    label {
      display: inline-block;
      width: 170px;
      font-weight: bold;
      font-size: 14px;
    }

    input,
    select {
      width: 250px;
      padding: 5px;
      font-size: 14px;
      box-sizing: border-box;
    }
  </style>
</head>
```

```

.btn-container {
    text-align: center;
    margin-top: 15px;
}

button {
    padding: 6px 14px;
    font-size: 14px;
    cursor: pointer;
    margin: 3px;
}

#billOutput {
    margin-top: 20px;
    padding: 15px;
    border: 1px dashed #333;
    background-color: #fafafa;
    display: none;
}

table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}

table,
th,
td {
    border: 1px solid #ddd;
    padding: 6px;
    font-size: 13px;
}

th {
    background-color: #eaeaea;
    text-align: left;
}
</style>
</head>

<body>

    <div class="container">
        <h2>Online Grocery Store</h2>
        <p style="text-align: center; font-size: 13px; color: #666;">Invoice & Bill Calculator -
24070521240</p>
        <hr>

        <form id="billForm">
            <!-- Customer Information -->
            <fieldset>
                <legend>Customer Details</legend>
                <div class="form-group">
                    <label for="custName">Customer Name:</label>
                    <input type="text" id="custName" required placeholder="Enter customer name">
                </div>
                <div class="form-group">
                    <label for="mobile">Mobile Number:</label>
                    <input type="tel" id="mobile" required placeholder="10-digit mobile number">
                </div>
                <div class="form-group">
                    <label for="invoiceNo">Invoice Number:</label>
                    <input type="text" id="invoiceNo" required placeholder="e.g. INV-1001">
                </div>
                <div class="form-group">
                    <label for="payMode">Payment Mode:</label>
                    <select id="payMode">
                        <option value="Cash">Cash</option>
                        <option value="Card">Card</option>
                        <option value="UPI">UPI</option>
                    </select>
                </div>
            </fieldset>
        </form>
    </div>

```

```

        </select>
    </div>
</fieldset>

<!-- Product Entry -->
<fieldset>
    <legend>Add Items</legend>
    <div class="form-group">
        <label for="productName">Product Name:</label>
        <input type="text" id="productName" placeholder="e.g. Rice / Sugar">
    </div>
    <div class="form-group">
        <label for="qty">Quantity (in Kg):</label>
        <input type="number" id="qty" step="0.01" min="0.1" placeholder="e.g. 5">
    </div>
    <div class="form-group">
        <label for="price">Price (₹):</label>
        <input type="number" id="price" step="0.01" min="0" placeholder="e.g. 300">
    </div>
    <div style="text-align: right;">
        <button type="button" onclick="addItem()" style="background-color: #e7e7e7;">+
Add Item</button>
    </div>

    <!-- Added Items List Table -->
    <div id="addedItemsSection" style="display: none; margin-top: 10px;">
        <h4 style="margin: 5px 0;">Added Items:</h4>
        <table>
            <thead>
                <tr>
                    <th>#</th>
                    <th>Item</th>
                    <th>Qty (Kg)</th>
                    <th>Price (₹)</th>
                    <th>Action</th>
                </tr>
            </thead>
            <tbody id="addedItemsBody"></tbody>
        </table>
    </div>
</fieldset>

<!-- Charges -->
<fieldset>
    <legend>Discounts & Charges</legend>
    <div class="form-group">
        <label for="discount">Discount (%):</label>
        <input type="number" id="discount" min="0" max="100" value="0">
    </div>
    <div class="form-group">
        <label for="packing">Packing Charges (₹):</label>
        <input type="number" id="packing" min="0" value="10">
    </div>
</fieldset>

<div class="btn-container">
    <button type="button" onclick="calculateBill()"
        style="background-color: #4CAF50; color: white;">Calculate Bill</button>
    <button type="reset" onclick="resetBill()">Reset</button>
</div>
</form>

<!-- Bill Invoice Output -->
<div id="billOutput">
    <h3>Invoice Summary</h3>
    <p><strong>Invoice No:</strong> <span id="outInvoice"></span></p>
    <p><strong>Customer:</strong> <span id="outName"></span> | <strong>Mobile:</strong>
        <span id="outMobile"></span></p>
    <p><strong>Payment Mode:</strong> <span id="outPayMode"></span></p>

```



```

<table>
  <thead>
    <tr>
      <th>Item Description</th>
      <th>Amount</th>
    </tr>
  </thead>
  <tbody id="outInvoiceItems"></tbody>
  <tfoot>
    <tr>
      <td>Subtotal Price</td>
      <td>₹<span id="outSubtotal"></span></td>
    </tr>
    <tr>
      <td>Discount (<span id="outDiscountPercent"></span>%)</td>
      <td>-₹<span id="outDiscountAmt"></span></td>
    </tr>
    <tr>
      <td>Packing Charges</td>
      <td>₹<span id="outPacking"></span></td>
    </tr>
    <tr style="font-weight: bold; background-color: #e6f7ff;">
      <td>Final Payable Amount</td>
      <td>₹<span id="outFinalTotal"></span></td>
    </tr>
  </tfoot>
</table>
</div>
</div>

<script>
  let itemList = [];

  function addItem() {
    let pName = document.getElementById("productName").value.trim();
    let qty = parseFloat(document.getElementById("qty").value);
    let price = parseFloat(document.getElementById("price").value);

    if (!pName || isNaN(qty) || qty <= 0 || isNaN(price) || price < 0) {
      alert("Please enter valid Product Name, Quantity, and Price!");
      return;
    }

    itemList.push({
      name: pName,
      qty: qty,
      price: price
    });

    // Clear item input fields
    document.getElementById("productName").value = "";
    document.getElementById("qty").value = "";
    document.getElementById("price").value = "";

    renderItemList();
  }

  function removeItem(index) {
    itemList.splice(index, 1);
    renderItemList();
  }

  function renderItemList() {
    let tbody = document.getElementById("addedItemsBody");
    tbody.innerHTML = "";

    itemList.forEach((item, index) => {
      tbody.innerHTML += `
        <tr>
          <td>${index + 1}</td>
          <td>${item.name}</td>
      `
    })
  }

```

```

        <td>${item.qty.toFixed(2)}</td>
        <td>₹${item.price.toFixed(2)}</td>
        <td><button type="button" onclick="removeItem(${index})" style="padding:1px
5px; color:red;">X</button></td>
    </tr>
    `;
    });

    document.getElementById("addedItemsSection").style.display = itemList.length > 0 ?
"block" : "none";
}

function calculateBill() {
    let name = document.getElementById("custName").value.trim();
    let mobile = document.getElementById("mobile").value.trim();
    let invoice = document.getElementById("invoiceNo").value.trim();
    let discount = parseFloat(document.getElementById("discount").value) || 0;
    let packing = parseFloat(document.getElementById("packing").value) || 0;
    let payMode = document.getElementById("payMode").value;

    if (!name || !mobile || !invoice) {
        alert("Please fill in Customer Name, Mobile Number, and Invoice Number!");
        return;
    }

    if (itemList.length === 0) {
        alert("Please add at least one item using the '+ Add Item' button!");
        return;
    }

    let subtotal = 0;
    let invoiceItemsHtml = "";

    itemList.forEach((item, i) => {
        subtotal += item.price;
        invoiceItemsHtml += `
            <tr>
                <td>${i + 1}. ${item.name} (${item.qty.toFixed(2)} Kg)</td>
                <td>₹${item.price.toFixed(2)}</td>
            </tr>
        `;
    });

    let discountAmt = (subtotal * discount) / 100;
    let priceAfterDiscount = subtotal - discountAmt;
    let finalTotal = priceAfterDiscount + packing;

    // Output values
    document.getElementById("outInvoice").innerText = invoice;
    document.getElementById("outName").innerText = name;
    document.getElementById("outMobile").innerText = mobile;
    document.getElementById("outPayMode").innerText = payMode;

    document.getElementById("outInvoiceItems").innerHTML = invoiceItemsHtml;
    document.getElementById("outSubtotal").innerText = subtotal.toFixed(2);
    document.getElementById("outDiscountPercent").innerText = discount;
    document.getElementById("outDiscountAmt").innerText = discountAmt.toFixed(2);
    document.getElementById("outPacking").innerText = packing.toFixed(2);
    document.getElementById("outFinalTotal").innerText = finalTotal.toFixed(2);

    document.getElementById("billOutput").style.display = "block";
}

function resetBill() {
    itemList = [];
    renderItemList();
    document.getElementById("billOutput").style.display = "none";
}
</script>
</body>
</html>

```

Output

The screenshot displays a web browser window with the title "Online Shopping - Bill Calculator". The URL bar shows the file path "C:/Users/icebear/Random/cs2/index.html". The browser's address bar includes "Ask Gemini" and "School" icons. The main content area is titled "Online Grocery Store" and "Invoice & Bill Calculator - 24070521240".

The form is divided into several sections:

- Customer Details:** Fields for Customer Name (Dustin), Mobile Number (987654321), Invoice Number (1001), and Payment Mode (Cash).
- Add Items:** Fields for Product Name (e.g. Rice / Sugar), Quantity (In Kg) (e.g. 5), and Price (₹) (e.g. 300). A "+ Add Item" button is present.
- Added Items:** A table listing the items added to the cart.
- Discounts & Charges:** Fields for Discount (%) (12) and Packing Charges (₹) (10). "Calculate Bill" and "Reset" buttons are located below these fields.
- Invoice Summary:** A summary of the invoice details and a table showing the item descriptions and amounts.

#	Item	Qty (Kg)	Price (₹)	Action
1	Rice	5.00	₹5000.00	X
2	Sugar	2.00	₹4000.00	X

Item Description	Amount
1. Rice (5.00 Kg)	₹5000.00
2. Sugar (2.00 Kg)	₹4000.00
Subtotal Price	₹9000.00
Discount (12%)	-₹1080.00
Packing Charges	+₹10.00
Final Payable Amount	₹7930.00

Result / Conclusion

The online shopping webpage was successfully developed using **HTML, CSS, and HTML5 tags**. The webpage displays the available products along with their details and provides a user-friendly interface for online shopping. The total bill amount was successfully calculated based on the selected products and their quantities.

Experiment 3

Title: Implement Control Structures and Form Validation; Create a Grading System Based on User-Entered Marks

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Student Marks & Grading System</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }

    h2,
    h3 {
      color: #333;
    }

    .form-group {
      margin-bottom: 10px;
    }

    label {
      display: inline-block;
      width: 120px;
    }

    input {
      padding: 5px;
    }

    button {
      padding: 6px 14px;
      margin-top: 5px;
      cursor: pointer;
    }

    .error {
      color: red;
      font-weight: bold;
      margin-top: 10px;
    }

    table {
      width: 100%;
      max-width: 500px;
      border-collapse: collapse;
      margin-top: 15px;
    }

    table,
```

```

th,
td {
    border: 1px solid #ccc;
}

th,
td {
    padding: 8px;
    text-align: left;
}

th {
    background-color: #f2f2f2;
}

.summary {
    margin-top: 15px;
    padding: 10px;
    background-color: #f9f9f9;
    border: 1px solid #ddd;
    max-width: 500px;
}
</style>
</head>

<body>

<h2>Student Grading System - 24070521240</h2>

<form id="gradeForm" onsubmit="return false;">
    <div class="form-group">
        <label for="subject">Subject Name:</label>
        <input type="text" id="subject" placeholder="e.g. Mathematics">
    </div>
    <div class="form-group">
        <label for="marks">Marks (0-100):</label>
        <input type="number" id="marks" placeholder="e.g. 85">
    </div>
    <button onclick="addSubject()">Add Subject</button>
</form>

<div id="errorMsg" class="error"></div>

<h3>Subject List</h3>
<table>
    <thead>
        <tr>
            <th>Subject</th>
            <th>Marks</th>
            <th>Grade</th>
        </tr>
    </thead>
    <tbody id="subjectTable">
        <!-- Dynamic Rows -->
    </tbody>
</table>

<div id="summary" class="summary" style="display: none;">
    <h3>Overall Summary</h3>
    <p><strong>Total Marks:</strong> <span id="totalMarks">0</span></p>
    <p><strong>Percentage:</strong> <span id="percentage">0</span>%</p>
    <p><strong>Overall Grade:</strong> <span id="overallGrade">-</span></p>
    <p><strong>Result:</strong> <span id="result">-</span></p>
</div>

<script>
    // Array to store subject objects
    let subjects = [];

    // Control Structure: Function using if-else statements to determine grade
    function calculateGrade(marks) {

```

```

    if (marks >= 90) {
        return 'A+';
    } else if (marks >= 80) {
        return 'A';
    } else if (marks >= 70) {
        return 'B';
    } else if (marks >= 60) {
        return 'C';
    } else if (marks >= 50) {
        return 'D';
    } else {
        return 'F';
    }
}

// Function for form validation and adding subject
function addSubject() {
    let subjectInput = document.getElementById("subject");
    let marksInput = document.getElementById("marks");
    let errorDiv = document.getElementById("errorMsg");

    let subjectName = subjectInput.value.trim();
    let marksValue = marksInput.value.trim();

    // Form Validation using Control Structures
    if (subjectName === "") {
        errorDiv.innerText = "Error: Subject name cannot be empty.";
        return;
    }

    if (marksValue === "" || isNaN(marksValue)) {
        errorDiv.innerText = "Error: Please enter valid numerical marks.";
        return;
    }

    let marks = parseFloat(marksValue);

    if (marks < 0 || marks > 100) {
        errorDiv.innerText = "Error: Marks must be between 0 and 100.";
        return;
    }

    // Clear previous error message if validation succeeds
    errorDiv.innerText = "";

    // Calculate grade for the subject
    let grade = calculateGrade(marks);

    // Add subject to list
    subjects.push({ name: subjectName, marks: marks, grade: grade });

    // Reset form fields
    subjectInput.value = "";
    marksInput.value = "";

    // Update table display and summary
    updateUI();
}

// Function to update the table and calculations using a loop
function updateUI() {
    let tableBody = document.getElementById("subjectTable");
    tableBody.innerHTML = "";

    let total = 0;

    // Control Structure: for loop to process all subjects
    for (let i = 0; i < subjects.length; i++) {
        total += subjects[i].marks;

        let row = `<tr>

```

```

        <td>${subjects[i].name}</td>
        <td>${subjects[i].marks}</td>
        <td>${subjects[i].grade}</td>
    </tr>`;
    tableBody.innerHTML += row;
}

// Calculate total percentage and overall grade
if (subjects.length > 0) {
    let percentage = total / subjects.length;
    let overallGrade = calculateGrade(percentage);
    let result = percentage >= 50 ? "PASS" : "FAIL";

    document.getElementById("totalMarks").innerText = total;
    document.getElementById("percentage").innerText = percentage.toFixed(2);
    document.getElementById("overallGrade").innerText = overallGrade;
    document.getElementById("result").innerText = result;

    document.getElementById("summary").style.display = "block";
}
}
</script>
</body>
</html>

```

Output




 File C:/Users/Icebear/Random/exp3/index.html

Student Grading System - 24070521240

Subject Name:

Marks (0-100):

Subject List

Subject	Marks	Grade
js	40	F
cn	69	C
cc	45	F

Overall Summary

Total Marks: 154

Percentage: 51.33%

Overall Grade: D

Result: PASS

Result / Conclusion

The experiment was performed successfully. Control structures (if, if...else, and if...else if...else) were used to determine grades based on user-entered marks. Form validation ensured that only valid marks within the range of **0 to 100** were accepted. The program correctly displayed the corresponding grade and prevented invalid input, demonstrating the practical use of decision-making statements and input validation in JavaScript.

Case Study 3

Title: Write a JavaScript program that accepts student marks and displays the scholarship category.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="refresh" content="0; url=ScholorshipEligibility.html">
  <title>Scholarship Eligibility</title>
</head>
<body>
  <p>Redirecting to <a href="ScholorshipEligibility.html">ScholorshipEligibility.html</a>...</p>
</body>
</html>

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Scholarship Eligibility Checker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
      background-color: #f4f4f4;
    }

    .container {
      max-width: 500px;
      background: #fff;
      padding: 20px;
      border: 1px solid #ccc;
      border-radius: 5px;
    }

    h2 {
      text-align: center;
      color: #333;
    }

    .form-group {
      margin-bottom: 15px;
    }

    label {
      display: block;
      margin-bottom: 5px;
      font-weight: bold;
    }

    input,
    select {
      width: 100%;
      padding: 8px;
      box-sizing: border-box;
    }

    button {
      width: 100%;
      padding: 10px;
      background-color: #007bff;

```

```

        color: white;
        border: none;
        cursor: pointer;
        font-size: 16px;
    }

    button:hover {
        background-color: #0056b3;
    }

    #result {
        margin-top: 20px;
        padding: 10px;
        border: 1px solid #ddd;
        background-color: #e9ecef;
        display: none;
    }
}
</style>
</head>

<body>

    <div class="container">
        <h2>Scholarship Eligibility Engine - 24070521240</h2>
        <form id="scholarshipForm" onsubmit="evaluateEligibility(event)">
            <div class="form-group">
                <label for="name">Student Name:</label>
                <input type="text" id="name" required placeholder="Enter student name">
            </div>

            <div class="form-group">
                <label for="score">Academic Score (%):</label>
                <input type="number" id="score" min="0" max="100" step="0.1" required
placeholder="e.g. 85">
            </div>

            <div class="form-group">
                <label for="income">Annual Family Income (₹):</label>
                <input type="number" id="income" min="0" required placeholder="e.g. 250000">
            </div>

            <div class="form-group">
                <label for="category">Demographic Category:</label>
                <select id="category" required>
                    <option value="">-- Select Category --</option>
                    <option value="General">General</option>
                    <option value="OBC">OBC</option>
                    <option value="SC/ST">SC / ST</option>
                </select>
            </div>

            <button type="submit">Check Eligibility</button>
        </form>

        <div id="result"></div>
    </div>

    <script>
        function evaluateEligibility(event) {
            event.preventDefault();

            let name = document.getElementById('name').value;
            let score = parseFloat(document.getElementById('score').value);
            let income = parseFloat(document.getElementById('income').value);
            let category = document.getElementById('category').value;

            let resultDiv = document.getElementById('result');
            let status = "";
            let scholarshipAmount = "";
            let color = "black";

```

```

// Conditional logic structure for eligibility matrix
if (score >= 90 && income <= 800000) {
    status = "Eligible for Full Merit Scholarship (100% Tuition Fee Waiver)";
    color = "green";
} else if (score >= 75 && income <= 300000) {
    status = "Eligible for Need-cum-Merit Scholarship (75% Fee Waiver)";
    color = "green";
} else if (category === "SC/ST" && score >= 60 && income <= 500000) {
    status = "Eligible for Special Category Support Scheme (60% Fee Waiver)";
    color = "green";
} else if (category === "OBC" && score >= 70 && income <= 400000) {
    status = "Eligible for Backward Class Assistance Grant (50% Fee Waiver)";
    color = "green";
} else if (score >= 60 && income <= 200000) {
    status = "Eligible for Financial Assistance Scheme (30% Fee Waiver)";
    color = "green";
} else {
    status = "Not Eligible for Current Scholarship Schemes.";
    color = "red";
}

resultDiv.style.display = "block";
resultDiv.style.color = color;
resultDiv.innerHTML = "<h3>Evaluation Result for " + name + ":</h3>" +
    "<p><strong>Category:</strong> " + category + "</p>" +
    "<p><strong>Academic Score:</strong> " + score + "%</p>" +
    "<p><strong>Annual Income:</strong> ₹" + income + "</p>" +
    "<p><strong>Status:</strong> " + status + "</p>";
}
</script>

</body>
</html>

```

Output

The image displays two screenshots of a web application titled "Scholarship Eligibility Engine - 24070521240". The application is accessed via a browser at the file path "C:/Users/icebear/Random/cs3/ScholarshipEligibility.html".

Top Screenshot (Eligible):

- Student Name:** Nancy
- Academic Score (%):** 99
- Annual Family Income (₹):** 250000
- Demographic Category:** OBC
- Check Eligibility** (button)
- Evaluation Result for Nancy:**
 - Category: OBC
 - Academic Score: 99%
 - Annual Income: ₹250000
 - Status: Eligible for Full Merit Scholarship (100% Tuition Fee Waiver)

Bottom Screenshot (Not Eligible):

- Student Name:** Nancy
- Academic Score (%):** 60
- Annual Family Income (₹):** 250000
- Demographic Category:** OBC
- Check Eligibility** (button)
- Evaluation Result for Nancy:**
 - Category: OBC
 - Academic Score: 60%
 - Annual Income: ₹250000
 - Status: Not Eligible for Current Scholarship Schemes.

Result / Conclusion

The JavaScript program was successfully developed to accept the student's marks and display the appropriate scholarship category based on the marks obtained. The program was tested with different marks and produced the expected scholarship category successfully.

Experiment 4

Title: Use Function Types, Scope, and Closures; Apply Try-Catch; Build a Palindrome Checker

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Palindrome Checker</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f9;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }

    .container {
      background: #fff;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 2px 5px rgba(0, 0, 0, 0.15);
      width: 320px;
      text-align: center;
    }

    input {
      width: 85%;
      padding: 8px;
      margin-bottom: 12px;
      border: 1px solid #ccc;
      border-radius: 4px;
    }

    button {
      padding: 8px 16px;
      background-color: #007bff;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }

    button:hover {
      background-color: #0056b3;
    }

    #result {
      margin-top: 15px;
    }
  </style>
</head>

<body>
  <div class="container">
    <input type="text" value="" />
    <button>Check</button>
    <div id="result"></div>
  </div>
</body>
</html>
```

```

        font-weight: bold;
    }
</style>
</head>

<body>

    <div class="container">
        <h2>Palindrome Checker - 24070521240</h2>
        <input type="text" id="userInput" placeholder="Enter word or phrase">
        <br>
        <button onclick="handleCheck()">Check</button>
        <div id="result"></div>
    </div>

    <script>
        // 1. GLOBAL SCOPE
        const appTitle = "Palindrome Checker App";

        // 2. CLOSURE (remembers total check count across function calls)
        function createCounter() {
            let count = 0; // Outer scope variable preserved by closure
            return function () {
                count++;
                return count;
            };
        }
        const getNextCount = createCounter();

        // 3. FUNCTION TYPES
        // Arrow Function Type: Cleans input string
        const cleanString = (str) => str.toLowerCase().replace(/[^a-z0-9]/g, '');

        // Function Expression Type: Validates input
        const validateInput = function (text) {
            if (!text || text.trim() === '') {
                throw new Error("Input cannot be empty!");
            }
        };

        // Function Declaration Type: Main click handler
        function handleCheck() {
            // BLOCK / LOCAL SCOPE
            const inputVal = document.getElementById("userInput").value;
            const resultDiv = document.getElementById("result");

            // 4. TRY...CATCH BLOCK
            try {
                // Validate input (throws error if empty)
                validateInput(inputVal);

                // Local processing variables
                const cleaned = cleanString(inputVal);
                const reversed = cleaned.split('').reverse().join('');
                const isPalindrome = (cleaned === reversed);

                // Using closure
                const currentCheckNum = getNextCount();

                if (isPalindrome) {
                    resultDiv.style.color = "green";
                    resultDiv.textContent = `${inputVal}` is a palindrome! (Check
                    #${currentCheckNum})`;
                } else {
                    resultDiv.style.color = "red";
                    resultDiv.textContent = `${inputVal}` is NOT a palindrome. (Check
                    #${currentCheckNum})`;
                }
            } catch (error) {
                // Catching thrown exception
            }
        }
    </script>

```

```
        resultDiv.style.color = "orange";  
        resultDiv.textContent = "Error: " + error.message;  
    }  
    }  
</script>  
</body>  
</html>
```

Output



Palindrome Checker - 24070521240

"abodefg" is NOT a palindrome. (Check #1)



Palindrome Checker - 24070521240

"abodcba" is a palindrome! (Check #2)

Result / Conclusion

The experiment was performed successfully. Different JavaScript function types, including function declarations, function expressions, and arrow functions, were implemented. Variable scope and closures were demonstrated to show how data is accessed and preserved within functions. A palindrome checker was developed that accepted user input, determined whether the entered text was a palindrome, and displayed the appropriate result.

Case Study 4

Title: ATM Card PIN Verification Using JavaScript Functions : An ATM system verifies whether a customer's PIN is a palindrome. If the PIN reads the same forward and backward, the bank displays a special security message. To implement this, create a function to reverse the entered PIN number and another function to check whether it is a palindrome. Use function declaration, function expression, arrow function, and demonstrate scope and closure concepts.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ATM Card PIN Verification</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      display: flex;
      justify-content: center;
      align-items: center;
      height: 100vh;
      margin: 0;
    }

    .card {
      background: #ffffff;
      padding: 25px;
      border-radius: 8px;
      border: 1px solid #ddd;
      width: 320px;
      box-shadow: 0 2px 5px rgba(0,0,0,0.1);
    }

    h2 {
      margin-top: 0;
      font-size: 20px;
      color: #333;
      text-align: center;
    }

    .form-group {
      margin-bottom: 15px;
    }

    label {
      display: block;
      margin-bottom: 5px;
      font-size: 14px;
      color: #555;
    }

    input {
      width: 100%;
      padding: 8px;
      box-sizing: border-box;
      border: 1px solid #ccc;
      border-radius: 4px;
      font-size: 16px;
    }
  </style>
</head>
</html>
```

```

    button {
        width: 100%;
        padding: 10px;
        background-color: #0066cc;
        color: white;
        border: none;
        border-radius: 4px;
        font-size: 15px;
        cursor: pointer;
    }

    button:hover {
        background-color: #004999;
    }

    #output {
        margin-top: 15px;
        padding: 10px;
        border-radius: 4px;
        font-size: 14px;
        line-height: 1.4;
    }

    .success {
        background-color: #d4edda;
        color: #155724;
        border: 1px solid #c3e6cb;
    }

    .normal {
        background-color: #e2e3e5;
        color: #383d41;
        border: 1px solid #d6d8db;
    }

    .error {
        background-color: #f8d7da;
        color: #721c24;
        border: 1px solid #f5c6cb;
    }

    .hidden {
        display: none;
    }
</style>
</head>
<body>

    <div class="card">
        <h2>ATM PIN Verification</h2>
        <div class="form-group">
            <label for="pinInput">Enter Customer PIN:</label>
            <input type="password" id="pinInput" placeholder="Enter PIN (e.g. 4334)" maxlength="8">
        </div>
        <button onclick="handleVerification()">Verify PIN</button>

        <div id="output" class="hidden"></div>
    </div>

    <script>
        // Global Scope variable demonstration
        const systemBank = "Secure Bank ATM";

        // 1. Function Declaration: Function to reverse the entered PIN number
        function reversePIN(pin) {
            // Converts pin to string, splits characters, reverses array, and joins back
            return pin.toString().split('').reverse().join('');
        }

        // 2. Function Expression: Function to check whether PIN is a palindrome
        const isPalindrome = function(pin) {

```

```

        const reversed = reversePIN(pin);
        return pin === reversed;
    };

    // 3. Arrow Function: Formats the security message
    const formatSecurityMessage = (isPal, attempts) => {
        if (isPal) {
            return `[Attempt #${attempts}] SPECIAL SECURITY NOTICE: PIN reads the same forward
and backward! (Verified by ${systemBank})`;
        } else {
            return `[Attempt #${attempts}] Standard PIN Verification: PIN is not a palindrome.`;
        }
    };

    // 4. Closure Concept: Creates a counter function that retains state of attempt count
    function createAttemptTracker() {
        let attempts = 0; // Private variable trapped inside closure scope

        return function() {
            attempts++;
            return attempts; // Inner function retains access to outer variable 'attempts'
        };
    }

    // Initialize closure instance
    const getAttemptCount = createAttemptTracker();

    // Main function (Demonstrates Local Scope concept)
    function handleVerification() {
        // Local Scope variables (accessible only within handleVerification function)
        const pinInput = document.getElementById('pinInput').value.trim();
        const outputDiv = document.getElementById('output');

        // Validation
        if (pinInput === "") {
            outputDiv.className = "error";
            outputDiv.innerText = "Error: Please enter a PIN number.";
            outputDiv.classList.remove("hidden");
            return;
        }

        // Call closure to get attempt number
        const attemptNumber = getAttemptCount();

        // Check palindrome using Function Expression
        const palindromeResult = isPalindrome(pinInput);

        // Format output using Arrow Function
        const message = formatSecurityMessage(palindromeResult, attemptNumber);

        // Display result
        outputDiv.classList.remove("hidden");
        if (palindromeResult) {
            outputDiv.className = "success";
        } else {
            outputDiv.className = "normal";
        }

        outputDiv.innerText = message;
    }
}
</script>
</body>
</html>

```

Output



Result / Conclusion

The ATM Card PIN Verification program was successfully implemented using JavaScript functions. The program reverses the entered PIN and checks whether it is a palindrome. **Function declaration, function expression, arrow function, scope, and closure concepts** were successfully demonstrated. The program correctly identifies whether the entered PIN reads the same forward and backward.

Experiment 5

Title: Apply Array Methods and Object Handling; Create a Cart Total Calculator with Discount Logic

Software/Tools Required

- Visual Studio Code (VS Code) or any Text Editor
- Web Browser (Google Chrome, Microsoft Edge, Mozilla Firefox, etc.)
- HTML
- JavaScript

Code

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <title>Shopping Cart Calculator</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 20px;
    }

    input {
      margin: 5px;
      padding: 5px;
    }

    table,
    th,
    td {
      border: 1px solid black;
      border-collapse: collapse;
      padding: 8px;
      margin-top: 10px;
    }

    .summary {
      margin-top: 15px;
      font-weight: bold;
    }
  </style>
</head>

<body>

  <h2>Shopping Cart Calculator - 24070521240</h2>

  <div>
    <input type="text" id="itemName" placeholder="Item Name">
    <input type="number" id="itemPrice" placeholder="Price">
    <input type="number" id="itemQty" placeholder="Quantity">
    <button onclick="addItem()" ">Add Item</button>
  </div>

  <div style="margin-top: 10px;">
    <label for="discount">Discount (%): </label>
    <input type="number" id="discount" value="0" min="0" max="100" oninput="updateCartUI()" ">
  </div>

  <h3>Cart Items</h3>
```

```

<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Price</th>
      <th>Quantity</th>
      <th>Total</th>
      <th>Action</th>
    </tr>
  </thead>
  <tbody id="cartTable"></tbody>
</table>

<div class="summary">
  <p>Subtotal: $<span id="subtotal">0.00</span></p>
  <p>Discount Amount: $<span id="discountAmt">0.00</span></p>
  <p>Grand Total: $<span id="grandTotal">0.00</span></p>
</div>

<script>
  // Array to store cart item objects
  let cart = [];

  // Add item (Object creation & Array push)
  function addItem() {
    const name = document.getElementById('itemName').value;
    const price = parseFloat(document.getElementById('itemPrice').value);
    const qty = parseInt(document.getElementById('itemQty').value);

    if (!name || isNaN(price) || isNaN(qty) || price <= 0 || qty <= 0) {
      alert("Please enter valid item details");
      return;
    }

    // Object handling: create an item object
    const item = {
      id: Date.now(),
      name: name,
      price: price,
      quantity: qty
    };

    cart.push(item);

    // Reset input fields
    document.getElementById('itemName').value = '';
    document.getElementById('itemPrice').value = '';
    document.getElementById('itemQty').value = '';

    updateCartUI();
  }

  // Array method: filter() to delete an item
  function removeItem(id) {
    cart = cart.filter(item => item.id !== id);
    updateCartUI();
  }

  // Update UI and calculate total using Array methods (map, reduce)
  function updateCartUI() {
    const tbody = document.getElementById('cartTable');
    tbody.innerHTML = '';

    // Array method: map() to build table rows
    cart.map(item => {
      const row = document.createElement('tr');
      const itemTotal = item.price * item.quantity;
      row.innerHTML = `
        <td>${item.name}</td>
        <td>${item.price.toFixed(2)}</td>
        <td>${item.quantity}</td>

```

```

        <td>${itemTotal.toFixed(2)}</td>
        <td><button onclick="removeItem(${item.id})">Delete</button></td>
    `;
    tbody.appendChild(row);
});

// Array method: reduce() to compute subtotal
const subtotal = cart.reduce((acc, item) => acc + (item.price * item.quantity), 0);

// Discount logic
const discountPercent = parseFloat(document.getElementById('discount').value) || 0;
const discountAmount = (subtotal * discountPercent) / 100;
const grandTotal = subtotal - discountAmount;

// Update total values in DOM
document.getElementById('subtotal').innerText = subtotal.toFixed(2);
document.getElementById('discountAmt').innerText = discountAmount.toFixed(2);
document.getElementById('grandTotal').innerText = grandTotal.toFixed(2);
}
</script>
</body>
</html>

```

Output

←
→
🔄
📁 File
C:/Users/Icebear/Random/exp5/index.html

Shopping Cart Calculator - 24070521240

Discount (%):

Cart Items

Name	Price	Quantity	Total	Action
Notebook	\$10.00	3	\$30.00	Delete
pen	\$5.00	200	\$1000.00	Delete

Subtotal: \$1030.00

Discount Amount: \$123.60

Grand Total: \$906.40

Result / Conclusion

The experiment was performed successfully. Arrays and objects were used to store and manage product information, while array methods such as `reduce()` were used to calculate the total cart value efficiently. Discount logic was implemented to determine the applicable discount based on the total purchase amount, and the final payable amount was displayed correctly. This experiment demonstrated the practical use of JavaScript arrays, objects, and array methods in developing a simple shopping cart application.